07｜实操课：顶层设计全流程演示

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

这是我们第一节实操课，核心目的就是展示 Claude Code 在本章节的真实操作过程，让你对其使用方法有一个更为直观和透彻的了解。

本节实操课将以文字 + 视频的方式进行，主要包含两部分：

> 第一部分是用文稿呈现我自己真实使用 AI Coding 的体验和看法，不是教程，是我自己的落地感受。同时也会回答下课程里大家比较感兴趣的几个问题。
>
> 第二部分是用录屏呈现顶层设计的完整实操过程，分为四个场景。这部分也会有参考文稿，不过更多细节会在视频里呈现，建议大家优先去观看视频。

我的真实体验和看法

目前就我个人而言，离开 Claude Code，我的生产力会严重下降。可能出乎你的意料，我是一个编码量特别大的人，但我跟 Claude Code 的协作，代码只是一部分。

这里总结一下我的哪些工作会交给 Claude Code 完成，具体如下：

让它给我做页面。我的开源项目需要运营管理平台，需要官网主页。我不擅长前端，开源社区找前端也非常麻烦。有了 Claude Code 后，这部分全部搞定。而且效果超级好，设计、风格、页面组成我都很满意。

让它给我写文档。我的流程一般是，我会跟 Claude Code 讨论某个观点。比如 AI Agent 通信，当前是怎么做的，有什么痛点，业界是怎样的，我会输出我的看法，然后让它给我整理成文章。

让它给我做市场调研。因为它有很多信息，比如最近我在研究 AI Agent 之间的通信，我会让它给我讲讲当前实际有什么问题，最新的 AI Infra 公司有在做类似的事情吗，当前的痛点是什么，各自的解决方案是什么。

让它给我做角色推演。我针对 AI Agent 通信的痛点出了一个开源方案。然后我让它站在资深基础架构师、一线开发工程师、算法工程师、Agent 自己（OpenClaw）的角度，发表看法。目的是获得各个角色的视角，避免自己技术自嗨，闭门造车。

跟它讨论技术方案。我会偶尔跟它讨论技术方案，但频率不高。可能我自己搞开发比较久，很多架构设计的事情我都有自己看法，它最多验证，我不会纯听它的。因为它总是给出看起来对、但是没那么适合我的方案。

跟它协作写代码。这是最主要的部分（可能也没那么多）。我自己比较保守，有点代码洁癖，所以我会区分项目的重要程度，确定跟它的合作深度。某些项目让它全部完成，某些项目只让它写部分。

让它给我当翻译。我的开源项目的官网英文文档，都是让它生成的，特别快。一句话：帮我生成这份文档的英文版。

闲聊。哈哈，这点其实蛮重要的。时不时会有一些 idea，我会快速问它的看法，让它发表观点，效率特别高。

我还用 Claude Code 做了好多事情，如果你感兴趣，我们可以在评论区讨论，或者大家特别有兴趣的话题，我可以在课程中做加餐。

下面我主要展开三个方面。

1. AI 协助做前端页面

我用 AI 协助搭建了开源项目的产品官网。我不是前端工程师，但 AI 让我能独立完成这些工作：帮我快速出页面、跟我探讨产品定位和文案措辞、做竞品和业界方案的调研。AI 在这个场景里的角色是协作者，不是执行者。后面的课程，也会分享哪些地方 AI 帮了大忙，哪些地方它给的东西不能直接用，必须自己判断。

2. 老项目改造

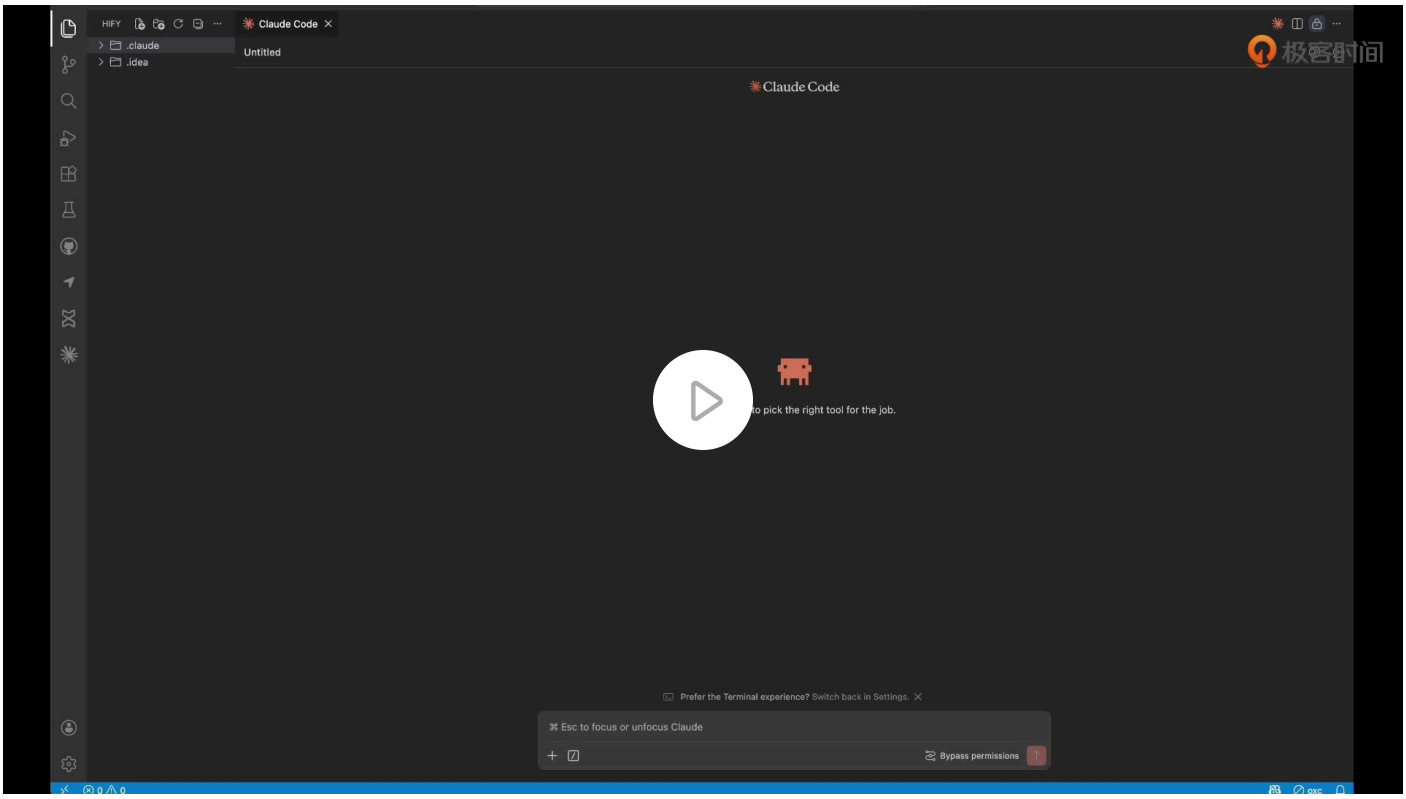
公司内部有一套 SDK 和 Web 项目，跑了很久，代码量大，架构固定。我尝试用 AI Coding 来做改造。老项目意味着既有代码、既有架构、既有约束，AI 不是从白纸开始，而是要在已有的框架里工作。给 AI 做好约束，以小任务执行为主。完整理解项目很难，先让 AI 通过小任务慢慢理解，上下文多了，就会越做越好。后面的课程中，我们也会分享哪些任务 AI 做得好、哪些地方它爱添乱、上下文长度和代码理解力的边界在哪，了解这些也会有帮助。

3. 新项目全部用 AI Coding 写

公司内部一个基于 ZooKeeper 的网关项目，从第一行代码开始就是纯 AI Coding 实现的。不是玩具项目，上线后承接了不小的流量，运行良好，至今没出过问题。这是跟老项目改造完全不同的体验，从零开始，你可以用 SDD 的方式定义规范，让 AI 在你设计的框架里工作。后面的课程中，我会分享新项目全程 AI Coding 的效率、质量、踩过的坑，以及“人定规范、AI 写代码”这个协作模式在生产环境的真实验证。

顶层设计全流程实操

这部分我们会用完整的屏录演示，让你看到真实的操作过程：我给了什么指令、Claude Code 回了什么、我怎么判断和追问、最终怎么定稿。



场景一：产品定义

目标：从梳理 Dify 功能全景到确定 Hify 的功能范围和技术栈。展示三问裁剪法的实际运用，以及你和 Claude Code 之间“它建议、你判断”的真实过程。

第一步：梳理 Dify 功能全景

指令：

```
帮我梳理 Dify（https://dify.ai）这个产品的核心功能模块，
按类别分组，每个模块用一两句话说明它做什么。
```

要点：

Claude Code 几秒钟给出八大模块的结构化清单

我拿它的输出和 Dify 官方文档对了一遍，修正了过时描述，补了 MCP 协议支持

强调“AI 加速信息收集，但你要验证”

第二步：功能取舍

指令：

```
我要基于 Dify 做一个简化版的 AI Agent 平台，叫 Hify。
约束条件：一个人开发，面向团队内部 20-50 人使用，本地部署。
请从刚才梳理的功能列表中，帮我判断哪些是必须做的核心功能，
哪些可以砍掉，给出每个的理由。
```

复制代码

要点：

Claude Code 给了三档分类（必须做 / 降级简化 / 砍掉），这个分档方式本身就有价值

重点展示 RAG 的判断，它建议保留，理由是“内部部署区别于 ChatGPT 的核心价值”。我被说服了，但降级：一期只支持 TXT

展示我否决权限体系的过程，50 人团队不需要

用三问裁剪法过一遍最终决策

第三步：技术选型

指令：

```
Hify 是一个 AI Agent 开发平台，一个人开发，本地部署，目标 20-50 人使用。
帮我对比以下技术方案的优劣：
1) Spring Boot + Vue  2) Go + React  3) Python FastAPI + React
重点考虑开发效率、生态成熟度、AI 领域 SDK 支持、运维复杂度。
```

复制代码

追问指令：

```
一个人做企业级后端，Spring Boot 和 FastAPI 在工程化能力上差距有多大？
```

要点：

> Claude Code 推荐 FastAPI，还给了“Java + Python 混合”的折中方案
>
> 我否决了混合方案，一个人维护两套技术栈是灾难
>
> 最终选 Spring Boot：AI 生态劣势存在，但工程化和技术栈统一性优先

场景二：应用架构设计

目标：确定模块化单体架构、Spring 代码组织规范、外部调用处理方案。展示和 Claude Code 的追问挑战过程。

第一步：应用架构选型

指令：

```
Hify 是一个 Spring Boot 单体应用，功能包括模型提供商管理、Agent 配置、
对话引擎、知识库 RAG、简版工作流、MCP 工具接入。一个人开发，一期 50 人使用，
但后续可能要扩到几千人。代码内部怎么组织？给我方案对比。
```

追问指令：

```
一个人维护六七个微服务，每个要独立部署、独立配置、独立监控，
精力消耗会不会太大
```

要点：

Claude Code 给了 3 个方案，推荐微服务

我用“一个人维护太重”这个现实约束否决，选模块化单体

展示最终的 Maven 模块划分（七个业务模块 + common）

第二步：代码组织规范

指令：

```
Hify 是模块化单体，用 Spring Boot + MyBatis-Plus。帮我定义代码组织规范，
覆盖：每个模块内部的分层结构、每一层的职责边界、跨模块调用的规则。
要求具体到 AI 能直接执行，不要模糊的描述。
```

复制代码

要点：

强调“这些规范是给 AI 写的，不是给人写的”

Controller 只做参数校验、跨模块走 Service 接口，人凭习惯知道，AI 不知道

展示规范写进 CLAUDE.md 的过程

第三步：外部调用设计

指令：

```
Hify 要调用多个外部 LLM API（OpenAI、Claude、Gemini、Ollama），
这些调用慢且不稳定。从线程管理、容错、超时、重试四个维度，给出完整的技术方案。
```

复制代码

追问指令：

```
流式响应用 SSE，Spring MVC 怎么处理？需不需要引入 WebFlux？
```

要点：

- Claude Code 建议 WebFlux，我否决，不增加技术栈复杂度
- SseEmitter + 独立线程池 + 120 秒超时，50 人够用
- 展示线程池隔离、熔断、超时、重试的完整方案写进 CLAUDE.md

场景三：部署架构与数据设计

目标：设计当前部署架构、预判性能瓶颈、规划扩展路径、定义数据模型和数据库规范。

第一步：当前部署架构

指令：

```
Hify 是模块化单体，技术栈 Spring Boot + Vue + MySQL + Redis + pgvector。
目标 50 人内部使用，生产环境用 Docker + K8s 部署。
帮我设计当前阶段的部署架构：有哪些组件、请求怎么流转、每个组件的职责是什么。
```

要点：

- Claude Code 给了组件职责表 + “当前做 vs 暂时跳过”对比表
- 重点展示“暂时跳过”的判断——单副本够用、MQ 是过度设计、不需要链路追踪

第二步：性能瓶颈预判

指令：

```
基于 Hify 当前的部署架构，帮我分析：这个系统的性能瓶颈可能在哪？
按严重程度排序，每个瓶颈给出触发条件和一期是否需要处理。
```

复制代码

要点：

Claude Code 给了 7 个瓶颈的排序表，结论是“一期只做两件事”

这张表就是你的系统软肋地图，现在不处理但心里有数

大部分工程师不会提前想这个，Claude Code 帮你几分钟画出来

第三步：数据库规范

指令：

复制代码

```
Hify 用 MySQL 8.x + pgvector。帮我定义数据库层面的性能规范，
覆盖：索引设计原则、大表预判和应对策略、分页查询注意事项、通用字段约定。
要求具体到 AI 建表时能直接执行。
```

要点：

展示 Claude Code 给的五条索引规则（带正反例代码）

重点展示“逻辑删除字段必须加进索引”和“游标分页”，这些不写进规范 AI 就不会遵守

展示 pgvector HNSW 索引规范

场景四：Claude Code 实战技巧

目标：演示 compact/clear 的实际操作效果、用业界规范喂 Claude Code 生成 CLAUDE.md、review 修改过程。

第一步：compact 和 clear 演示

操作：

展示一个对话已经很长的场景（前面几个场景的对话累积）

输入 /compact，展示压缩前后的上下文变化

输入 /clear，展示清空后 CLAUDE.md 自动加载的效果

新对话里输入一条简单指令，验证 Claude Code 还知道项目规范

要点：

每个任务完成后 commit + clear 是最佳实践

CLAUDE.md 自动加载，项目规范不丢

compact 适合任务中途上下文快满了，clear 适合切换任务

第二步：业界规范喂入

指令：

复制代码

```
我在做一个 Spring Boot 项目，请基于阿里巴巴 Java 开发手册，
帮我提炼出最关键的 20 条编码规范，写成 CLAUDE.md 可以直接用的格式。
重点覆盖命名、异常处理、日志、并发这几个方面。
不要照搬原文，要精简到 AI 能直接执行。
```

要点：

Claude Code 几分钟生成精简版，不是照搬原文而是提炼关键条目

你不需要读完几十页规范，让 AI 消化后帮你提炼

展示 review 过程：删掉不适用的条目、调整措辞

第三步：生成完整 CLAUDE.md

指令：

```
以下是 Hify 项目前期做的所有决策：[贴上所有片段]
另外请基于阿里巴巴 Java 开发手册，补充编码规范部分。
请帮我合并生成一份完整的 CLAUDE.md。
```

要点：

展示 Claude Code 合并后的完整版（滚动浏览）

指出 3 处需要人工修改的地方：行为指令措辞太软、“不做什么”有遗漏、多了不必要的 logback 配置

修改定稿后 commit

到这里 CLAUDE.md 完整版就位，Claude Code 对 Hify 的全部认知就绪

生成的 Claude.md

```
# Hify 项目开发规范

## 项目概览

Hify 是一个简化版内部 AI Agent 平台，基于 Dify 思路设计。

- **团队规模**：1 人开发，20-50 人内部使用，本地部署
- **技术栈**：Spring Boot + MyBatis-Plus + Vue + MySQL 8.x + Redis + pgvector
- **架构模式**：模块化单体（Modular Monolith），代码边界清晰，可平滑拆分为微服务

---

## 核心功能模块（MVP 范围）

| 模块 | 说明 |
|------|------|
| 模型管理（model） | 管理 OpenAI / Claude / Gemini / Ollama 等 LLM 提供商配置，支持连通
| Agent 配置（agent） | 配置 Agent 名称、系统提示词、绑定模型、关联知识库和 MCP 工具 |
| 对话引擎（conversation） | 多轮对话、历史记录、SSE 流式响应 |
| 知识库 RAG（knowledge） | 文档上传 → 异步向量化 → pgvector 余弦搜索 → 注入 LLM 上下文 |
| 简版工作流（workflow） | 顺序节点执行：开始 → LLM → 条件分支 → 工具调用 → 结束 |
| MCP 工具接入（mcp） | 接入外部 MCP 工具，供 Agent 和工作流调用 |

```

砍掉的功能：多租户、自定义插件市场、实时协作、企业 SSO、精细化权限控制、数据集版本管理。

代码组织规范

包结构

```
com.hify
├── common/
│   ├── config/          # MybatisPlusConfig, JacksonConfig, AsyncConfig, ThreadP
│   ├── exception/       # BizException, ErrorCode, GlobalExceptionHandler
│   ├── web/             # Result<T>, PageResult<T>
│   └── util/            # JsonUtil, DateUtil
├── modules/
│   ├── model/           # LLM 提供商管理
│   ├── agent/           # Agent 配置
│   ├── conversation/    # 对话引擎
│   ├── knowledge/       # 知识库 RAG
│   ├── workflow/        # 简版工作流
│   └── mcp/             # MCP 工具接入
└── HifyApplication.java
```

每个模块内部四层结构：

```
modules/{module}/
├── api/        # 对外暴露的接口（interface），供其他模块调用
├── domain/     # 业务逻辑：Service 实现、领域对象、Factory、Repository 接口
├── infra/      # 基础设施：Mapper、RepositoryImpl、外部 API 客户端、config
└── web/        # Controller，只处理 HTTP 层
```

各层职责边界

层	职责	禁止
web/	接收请求、参数校验（@Valid）、调用本模块 api/ 接口、返回 Result<T>	直接调用其他
api/	定义跨模块调用的 interface 和 DTO	包含业务逻辑实现
domain/	业务逻辑、领域对象、事务边界（@Transactional）	直接依赖 Mapper、依赖 web 层
infra/	Mapper、RepositoryImpl（PO ↔ 领域对象转换）、外部调用	包含业务逻辑

跨模块调用规则

- **只能**通过目标模块的 `api/` 接口调用，禁止直接 import 其他模块的 `domain/` 或 `infra/
- 跨模块传递使用 `api/` 包下定义的 DTO，不传递 PO 或领域对象
- 循环依赖视为架构错误，立即重构

```java
// 正确：agent 模块通过 ModelService（api/ 接口）调用 model 模块
@Service
```

```
@RequiredArgsConstructor
public class AgentServiceImpl implements AgentService {
    private final ModelService modelService; // 来自 model 模块的 api/ 接口
}
```

LLM 调用规范

线程池配置

```java
// llm-pool: 非流式调用（阻塞等待完整响应）
@Bean("llmExecutor")
public ThreadPoolExecutor llmExecutor() {
    return new ThreadPoolExecutor(20, 50, 60L, TimeUnit.SECONDS,
        new LinkedBlockingQueue<>(100),
        new ThreadFactoryBuilder().setNameFormat("llm-pool-%d").setDaemon(true).b
        new ThreadPoolExecutor.CallerRunsPolicy()  // 满载时调用方线程执行，不丢任务
    );
}

// llm-stream: 流式 SSE 调用（长连接）
@Bean("llmStreamExecutor")
public ThreadPoolExecutor llmStreamExecutor() {
    return new ThreadPoolExecutor(30, 80, 60L, TimeUnit.SECONDS,
        new LinkedBlockingQueue<>(50),
        new ThreadFactoryBuilder().setNameFormat("llm-stream-%d").setDaemon(true)
        new AbortPolicy()  // 流式超限直接拒绝，由上层返回 503
    );
}
```

OkHttpClient 配置

```java
// 非流式：有 readTimeout
@Bean("standardLlmClient")
public OkHttpClient standardLlmClient() {
    return new OkHttpClient.Builder()
        .connectTimeout(5, TimeUnit.SECONDS)
        .readTimeout(120, TimeUnit.SECONDS)
        .writeTimeout(30, TimeUnit.SECONDS)
        .connectionPool(new ConnectionPool(20, 5, TimeUnit.MINUTES))
        .addInterceptor(new LoggingInterceptor())
        .build();
}

```

```
// 流式：readTimeout 设为 0（SSE 不能有读超时）
@Bean("streamLlmClient")
public OkHttpClient streamLlmClient() {
    return new OkHttpClient.Builder()
        .connectTimeout(5, TimeUnit.SECONDS)
        .readTimeout(0, TimeUnit.SECONDS)
        .writeTimeout(30, TimeUnit.SECONDS)
        .build();
}
```

超时层次（三层保护）

1. OkHttp connectTimeout = 5s（TCP 握手超时）
2. OkHttp readTimeout = 120s（单次读取超时，仅非流式）
3. CompletableFuture.get(90, TimeUnit.SECONDS)（总体超时兜底）

重试策略（Resilience4j）

- 普通 LLM：最多 3 次，初始等待 500ms，指数退避 2x，最大等待 10s
- Ollama（本地）：最多 5 次，初始等待 2s
- 仅对网络异常和 5xx 重试，4xx（参数错误）不重试

熔断器配置

```yaml
# COUNT_BASED 滑动窗口，20 次请求内失败率 >50% 触发熔断
# 慢调用（>30s）超过 80% 也触发熔断
# 熔断后等待 30s 进入 half-open，放行 5 次探测
failure-rate-threshold: 50
slow-call-duration-threshold: 30s
slow-call-rate-threshold: 80
wait-duration-in-open-state: 30s
permitted-calls-in-half-open-state: 5
```

Fallback 路由

```yaml
hify.llm.fallback:
  openai: ollama
  claude: openai
  gemini: ollama
```

主 Provider 熔断或异常时自动切换 fallback，fallback 失败则抛出 BizException。

```
## 部署架构

用户浏览器
    │
    ▼
Ingress Nginx（L7 负载均衡 + SSL 终止 + SSE 支持）
    │
    ├──► hify-frontend（Vue SPA，Nginx 静态文件服务，2 副本）
    │
    └──► hify-backend（Spring Boot，2 副本）
              │
              ├──► MySQL 8.x（主数据存储）
              ├──► Redis（Session / 缓存 / 限流）
              └──► PostgreSQL + pgvector（向量存储）

**Ingress 关键配置（SSE 必须）**：

```yaml
nginx.ingress.kubernetes.io/proxy-read-timeout: "300"
nginx.ingress.kubernetes.io/proxy-buffering: "off"
nginx.ingress.kubernetes.io/limit-rps: "20"
```

Backend 容器规格：requests 512Mi/250m，limits 1Gi/1000m，replicas=2

JVM 启动参数：

```dockerfile
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-XX:+UseG1GC",
            "-Djava.security.egd=file:/dev/./urandom", "-jar", "app.jar"]
```

数据库规范

MySQL 通用字段约定

每张表必须包含以下字段：

```sql
id          BIGINT          NOT NULL AUTO_INCREMENT,  -- 主键，禁用 UUID
created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRE
deleted     TINYINT(1)      NOT NULL DEFAULT 0,       -- 逻辑删除标志
PRIMARY KEY (id)
```

- 字符集：`utf8mb4`，排序规则：`utf8mb4_unicode_ci`
```

```
- 禁用 `VARCHAR` 无长度约束，text 类 content 字段用 `MEDIUMTEXT`
- 金额用 `DECIMAL(19,4)`，禁用 `FLOAT/DOUBLE`
- 布尔用 `TINYINT(1)`，不用 `BIT`

### 索引设计原则

1. **区分度低的字段不单独建索引**（如 deleted、status 枚举），必须与高区分度字段组合
2. **组合索引遵循最左前缀**：等值查询字段在左，范围查询字段在右
3. **查询条件中含 `deleted`**，必须将 `deleted` 纳入索引
4. **每表索引不超过 5 个**（含主键），写多读少的表控制在 3 个以内
5. **禁止在 `TEXT/BLOB` 类型字段上建普通索引**，需要时建前缀索引或全文索引

```sql
-- 正确示例：conversation_id 高区分度在左，deleted 次之，created_at 范围在右
INDEX idx_conv_created (conversation_id, deleted, created_at)
```

大表处理策略

判断为大表的阈值：行数 > 500 万 或 数据量 > 2GB

场景	策略
t_message	按 conversation_id 分区，或按月归档冷数据
知识库向量表	ivfflat 索引，lists = sqrt(行数)
日志类表	只保留 90 天，定期 DELETE + OPTIMIZE TABLE

分页查询规范

- **禁止** `LIMIT offset, size` 深分页（offset > 1000 全表扫描）
- 对话记录类使用**游标分页**：

```sql
SELECT id, role, content, created_at FROM t_message
WHERE conversation_id = ?
  AND deleted = 0
  AND (created_at < ? OR (created_at = ? AND id < ?))
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

- 管理后台必须分页时，用 `WHERE id > lastId LIMIT size` 替代 offset

pgvector 索引规范

```sql
-- 余弦相似度索引，lists 值 = sqrt(总行数)，行数 <10 万时 lists=100
CREATE INDEX idx_embedding_ivfflat ON knowledge_embedding
USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
```
```

```
-- 查询时设置 probes，精度和速度平衡
SET ivfflat.probes = 10;
SELECT * FROM knowledge_embedding
ORDER BY embedding <=> '[...]'::vector LIMIT 5;
```

索引检测措施

开发阶段：启用 p6spy，拦截执行 >10ms 的查询自动 EXPLAIN，type=ALL 时打印警告日志。

CI 阶段：关键查询写 `IndexCoverageTest`，EXPLAIN 结果中 type=ALL 则测试失败，阻断合并

生产阶段：定期查询 `performance_schema.events_statements_summary_by_digest`，找出

```sql
SELECT digest_text, count_star AS 执行次数, sum_no_index_used AS 未用索引次数
FROM performance_schema.events_statements_summary_by_digest
WHERE sum_no_index_used > 0
ORDER BY sum_no_index_used DESC LIMIT 20;
```

编码规范（基于阿里巴巴 Java 开发手册）

命名

1. **类名用 UpperCamelCase**，方法名、变量名用 lowerCamelCase，常量用 UPPER_SNAKE_CASE
2. **禁止用拼音或拼音缩写**命名，禁止单字母变量（循环变量 `i/j/k` 除外）。
3. **方法名体现动词**：查询用 `get/list/query`，修改用 `update`，删除用 `delete/remove`
4. **Service 接口不加 I 前缀**，实现类加 `Impl` 后缀（`AgentService` + `AgentServiceIm
5. **数据库表名用 `t_` 前缀**，列名用 snake_case；PO 类用 `Po` 后缀，DTO 用 `Dto`/`Requ

异常处理

6. **禁止 catch 后 `e.printStackTrace()` 或空 catch**，必须记录日志或向上抛出。
7. **业务异常统一抛 `BizException(ErrorCode)`**，不用 RuntimeException 传递业务语义。
8. **只在顶层（GlobalExceptionHandler）处理并转换为 HTTP 响应**，中间层不捕获再包装。
9. **finally 块不写 return**，不在 finally 中抛出新异常（会吞掉原始异常）。
10. **NPE 防御**：方法返回值优先返回空集合（`Collections.emptyList()`）而非 null，接口入参

日志

11. **使用 SLF4J 接口 + Logback 实现**，类中用 `@Slf4j`（Lombok），禁止用 `System.out.
12. **禁止在循环体内打日志**，高频路径只在异常分支记录。
13. **占位符格式 `log.info("xxx {}", var)`**，禁止字符串拼接（避免无效 toString 开销）。
14. **日志分级约定**：DEBUG=详细调试，INFO=关键业务节点，WARN=可恢复异常或配置缺失，ERROR=需
15. **LLM 调用必须记录**：provider、model、耗时、token 数、是否命中缓存，便于成本分析。

```
318
319 ### 并发
320
321 16. **线程池必须显式创建**（`ThreadPoolExecutor`），禁止用 `Executors.newFixedThreadPo
322 17. **ThreadLocal 用完必须 `remove()`**，防止线程池场景下数据泄漏。
323 18. **加锁粒度最小化**：只锁共享变量操作，不锁 I/O 和 LLM 调用；优先用 `ReentrantLock` 替代
324 19. **单例 Bean 的成员变量必须是线程安全的**：无状态 Service 天然安全；有状态则用 `ThreadLo
325 20. **`CompletableFuture` 异步调用必须指定线程池**（`supplyAsync(task, llmExecutor)`）
326
327 ---
328
329 ## 性能瓶颈优先级（一期处理清单）
330
331 | 级别 | 瓶颈 | 一期处理方式 |
332 |------|------|--------------|
333 | P0 | LLM API 延迟高（3-30s） | 线程隔离 + 熔断 + Fallback（已设计） |
334 | P0 | 向量检索无索引全表扫描 | 建 ivfflat 索引（建表时必须创建） |
335 | P1 | 对话消息深分页 | 游标分页（禁止 LIMIT offset） |
336 | P1 | N+1 查询 | MyBatis-Plus 批量查询，禁止循环单查 |
337 | P2 | 连接池耗尽 | HikariCP 配置：maximumPoolSize=20，connectionTimeout=3000ms |
338 | 延后 | 静态资源未压缩 | Nginx gzip，流量大时处理 |
339 | 延后 | JVM GC 停顿 | G1GC 已启用，暂不调优 |
340
```

总结

两部分内容讲完，顶层设计篇正式结束。

第一部分，不只是几个场景的体验分享，更重要的是让你看到 AI Coding 的使用面远比“写代码”宽。做页面、写文档、市场调研、角色推演、技术讨论、翻译、闲聊……它是一个全方位的协作者。但它在不同场景下的表现差异很大，关键是你怎么用。

第二部分，四个场景走完，回顾一下你手上现在有什么：

一份完整的产品定义（做什么、不做什么、做到什么程度）

一套应用架构（模块划分、代码组织规范、外部调用处理）

一套系统架构（部署形态、性能瓶颈地图、三阶段扩展路径、数据库规范）

一份完整的 CLAUDE.md。

到这，我们没写一行业务代码，但所有决策都已经落成文字。从下一讲开始，Claude Code 就会在这些约束下写代码，它会清楚地知道：模块怎么分、代码放哪里、接口什么格式、索引怎么建、什么不能做。这就是 SDD 的威力，规范定得越清楚，后面写代码越快越稳。

期待你与我分享自己的使用体验。如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (17)

最新　精选



威
2026-04-10 来自广东

老师你好，后会不会有一些例子，针对现有的项目，利用AI来增加新功能或者修复bug，目前这方面的例子我看不多，都是从一些新项目开始的

> 作者回复: 这个有在考虑。主要是老项目改造其实细节特别多，是一个体系，它的思考逻辑跟新项目开发还不太一样，几节课讲不完。就讲几个例子的话，会感觉很单一，有种人浮于事的感觉。
> 其实我个人在思考，如果这门课的叙事节奏和主题能让大家学到东西，就可以考虑看是不是出第二门课，接着这门，专门讲老项目改造和开发的。看看后面的反馈呗，目的还是希望目的是让大家能学到东西。

4



s(t)
2026-07-01 来自上海

老师有个疑问就是我用codex+gpt5.5跑下来 内容和你的可能不太一样.这个有影响吗？我问了大模型，意思就是这只是给ai的一个综合性的指导，具体的会等到做的时候再完善细节.所以不太一样无所谓，到时候做的时候把没涉及到的补上就行了 是这样吗

> 作者回复: 不同的模型跑下来肯定是不一样的结果，甚至同一个模型跑下来也有细微的差别。
> 不过没有影响的。就像你说的：具体的会等到做的时候再完善细节就可以了。

关键是我们知道要做什么，然后做的方向是什么就可以了。细节可以慢慢调整。

3



庄周梦蝶
2026-04-20 来自浙江

老师，老项目改造也能实操讲解一下吗

作者回复: 可以的，这块的内容已经在规划了哈。敬请期待～

1



james.d
2026-04-08 来自四川

请教：在真实的工业级项目中，产品定义或者业务知识库需要细化到什么程度，系统，领域，功能，场景，页面，每一层都需要构建么?

作者回复: 这是个很大的问题了。核心就是看你要用这个知识库来解决问题。另外每个公司对不同业务线的资料可能会有保密措施。原则来看，知识库肯定越细，AI会执行的越好。但是在真实场景中，会遇到各种安全、组织架构等实际问题。

但是不用每一层都构建，而是构建一个总的，因为分的太细，AI 拿到的context也会割裂，做不好一个完整的全链路任务。

1



jimmy
2026-06-21 来自广东

老师好，请问除了claude的模型，国内的minimax m3和qianwen，glm，哪个相对体感比较好

作者回复: 这个我没用过诶，看一下评论区有没有大佬可以回答的～

共 2 条评论>



木木小王子
2026-06-02 来自北京

老师，你底层用的哪个基座大模型？ 我按照老师的操作来，结果差距还是有点大，我用的qwen3.7

作者回复: 我用的是claude 4.7 sonnet哈。感觉qwen的能力貌似还是有点不够。我没用过qwen，感觉可能需要多次提示。






禾子先生
2026-05-07 来自上海

之前一直在想，为什么给claude发那些问题指令，这节课看后豁然开朗。
同时也在思考另外一个问题，这里和claude交互还是非常多，难道就没有一些工具来将其中一些应该会或关键点自动提问和补充吗?

作者回复: 应该会或关键点自动提问和补充。
好问题啊，好创意。我去找找学习下，理论上应该要有的。我还真没从这个角度思考过问题。太有创意了。
如果评论区有人知道有啥工具，也可以推荐给我们～～

共 3 条评论>




菲尔德
2026-04-28 来自贵州

上一讲的 接口设计规范 这里不用写嘛

作者回复: 接口规范在：06 | Claude Code 实战必备：CLAUDE.md、Skills 与工程规范。 这里面有讲的






cricket1981
2026-04-12 来自上海

前端项目如何跟claude沟通设计开发？需要自己画原型图吗?

作者回复: 我自己一般有两个做法：
1. 简单的一句话描述，让AI 自由发挥，我感觉都会发挥的挺好。也有可能主要是因为我做的主要是后

台管理系统

2. 用https://excalidraw.com/ 画个布局图，仍给cc。

不过我是野路子，不是专业前端。我的目标是，好看就行。



巴马

2026-04-10 来自四川

老师，您的claude code是集成在什么工具里面的呢？ 我是用的传统的CLI

作者回复: 我是vscode 插件/idea 插件集成的，我很少用cli，实在用不来